

Lab 2: Text Extraction and Cleanup

Back-translation, HTML, a PDF, and a short preprocess chain.

Work in a **Jupyter** notebook. Number the exercises. Do notes **2.1** through **2.3** first. You will need `BackTranslation` (or `googletrans`), `beautifulsoup4`, `requests`, a PDF library (`pypdf` or `pdfminer.six`), and `nltk`.

Download [sample.pdf](#) into the same folder as the notebook (or use a relative path).

When you are done: **File** → **Download as** → **HTML**, then upload the HTML on Canvas. Save the notebook before you export.

```
import nltk
from nltk.stem import PorterStemmer
from nltk.stem import WordNetLemmatizer
from nltk.tokenize import word_tokenize
from nltk.corpus import stopwords

nltk.download("punkt")
nltk.download("wordnet")
nltk.download("stopwords")
```

If `word_tokenize` asks for `punkt_tab`, run `nltk.download("punkt_tab")` as well.

1. Back translation (1 point)

Use [BackTranslation](#) (it wraps Google Translate).

1. Pick one English sentence of your own.
2. Back-translate it through at least **three** intermediate languages. Print the source, the intermediate text, and the English that comes back.
3. Say in a short comment which language pair changed the sentence the most, and whether the label (if this were training data) would still be safe.

```
# pip install BackTranslation
from BackTranslation import BackTranslation

trans = BackTranslation(url=["translate.google.com", "translate.google.gr"])
result = trans.translate("I love NLP", src="en", tmp="el")
print(result.source_text)
print(result.tran_text)
print(result.result_text)
```

`tmp` is the pivot language (Greek is `"el"`). `trans.searchLanguage("Greek")` looks up codes.

Network calls can fail. If Translate rate-limits you, wait and retry, or switch the pivot. Do not paste a screenshot of someone else's output.

2. HTML parsing and cleanup (1 point)

Use [Beautiful Soup](#) and `requests`.

1. Open the Stack Overflow page for the question **“What is the module/method used to get the current time?”** Fetch the HTML. Extract the **question text** and the **best / accepted answer** text. Print both. (Find the tags/classes on that page; do not dump the whole document.)
2. Download this file to your local directory and confirm it exists:

`https://zoisboukouvalas.github.io/COVID19_Twitter_Dataset.xlsx`

`requests.get` plus writing `response.content` is enough. Print the saved path and file size.

Respect the sites: one request for the question page, one for the spreadsheet. No crawling loop.

3. Text from a PDF (1 point)

Open [sample.pdf](#). Extract the plain text with `pypdf` or `PDFMiner`. Print the extracted string.

If the extract looks broken (missing words, odd spaces), say so in a comment. That is part of the exercise.

```
from pypdf import PdfReader

reader = PdfReader("sample.pdf")
text = "".join(page.extract_text() or "" for page in reader.pages)
print(text)
```

4. Text pre-processing (2 points)

Use this corpus (copy it into the notebook):

```
Need to finalize the demo corpus which will be used for this notebook & should be done soon !!. It
should be done by the ending of this month. But will it? This notebook has been run 4 times !!
```

1. Lowercase the corpus.
 2. Remove digits, punctuation, and trailing whitespace.
 3. Tokenize with NLTK (or spaCy) and drop English stop words.
-

4. Stem the remaining tokens with `PorterStemmer`. Print the list.

5. Lemmatize the remaining tokens with `WordNetLemmatizer`. Print the list.

Stemming and lemmatization should run on the **same** cleaned tokens so you can compare them. `4` should be gone after step 2; `!!` should not be a token after step 3.